

Para crear un middleware que maneje errores en Express, sigue estos pasos:

1. **Definir la función del middleware:** Crea una función que reciba `error`, `req`, `res`, y `next`.
2. **Obtener el status code:** Extrae `error.statusCode` o asigna 500 como valor por defecto.
3. **Preparar el mensaje:** Captura `error.message` o usa un mensaje por defecto como "Ocurrió un error inesperado".
4. **Loggear el error:** Utiliza `console.error` para registrar detalles del error.
5. **Enviar respuesta:** Responde con `res.status(statusCode).json({ status: 'error', message, stack: process.env.NODE_ENV === 'development' ? error.stack : undefined })`.
6. **Integrar el middleware:** Usa `app.use(middlewareFunction)` después de definir tus rutas.

Esto te dará un manejo centralizado de errores en tu API.

En el ejemplo que se muestra, el middleware de manejo de errores (`errorHandler`) parece no ejecutarse correctamente, ya que en ninguna de las ejecuciones se ve la estructura JSON definida dentro de él.

Esto ocurre porque `app.use(errorHandler)` debe colocarse **después de todas las rutas**. De esa forma, cuando se produzca un error y se llame a `next(err)`, Express buscará el siguiente middleware que acepte cuatro parámetros (`err`, `req`, `res`, `next`), reconocerá que se trata de `errorHandler` y lo ejecutará, devolviendo la respuesta JSON esperada.

Para ocultar detalles técnicos de errores en una API desarrollada con Express, debes configurar la variable de entorno `NODE_ENV` a `production`. Esto asegura que los mensajes de error devueltos al usuario sean genéricos, mostrando solo un mensaje como "Internal Server Error", mientras que en el entorno de desarrollo puedes mostrar detalles específicos para facilitar la depuración. Esto es parte de las buenas prácticas en el manejo de errores en aplicaciones web.